

Research component 1

Research Component 1

Semantic OCR Post-Correction for Sinhala–English Financial Documents

1. Component Overview

Component Name

Semantic OCR Post-Correction Module

Position in System

- **First intelligence layer** in the ingestion pipeline
- Runs **after rasterization** and **before layout analysis**

Core Responsibility

Convert noisy OCR output from Sinhala–English financial documents into:

- **Semantically correct text**
- **Numerically immutable text**
- **Bounding-box aligned text units**

2. Problem Definition

2.1 Practical Problem

OCR engines (SuryaOCR) operate at a **pixel-to-character level** and lack semantic understanding. In Sri Lankan SME documents, this causes systematic failures.

2.2 Observed Failure Modes

A. Sinhala Ligature Corruption

Example:

Ground truth : ඉන්වොයිස්

OCR output : ඉන්වයිස්

Cause:

- Dense ligatures
- Minor pixel loss → catastrophic word corruption

B. Legacy Font PDF Noise

- Non-Unicode Sinhala fonts (e.g., FMAbhaya)
- Direct text extraction fails
- Rasterization fixes encoding but introduces pixelation artifacts

C. Dangerous Numeric Ambiguity

Example:

OCR: 500.00

Human: 500.00

! LLM correction **must not modify numbers**, even if it “looks correct”.

3. Design Goals

Goal	Description
Semantic Recovery	Fix words using context
Numeric Safety	Never alter digits
Auditability	Every correction must be traceable
Cost Efficiency	Must run on Colab T4
Language Support	Sinhala + English

4. Inputs and Outputs

4.1 Inputs

Input Artifacts

Name	Type
page_images[]	PNG images (300 DPI)
tenant_id	string
document_id	string

4.2 Outputs

Primary Output

final_safe_boxes.json

```
{
  "page": 1,
  "boxes": [
    {
      "text": "ඉන්වෙන්ටරි අංකය",
      "bbox": [120, 80, 380, 120],
      "confidence": 0.71,
      "locked_digits": false,
      "source": "ai_corrected"
    }
  ]
}
```

Secondary Outputs

- raw_ocr_boxes.json
- ai_corrected_boxes.json
- plain_text.txt
- Quality_report.json

5. Internal Architecture

Image



SeuyaOCR



Raw OCR Boxes



LLM Semantic Correction



Numeric Safe-Guard



Final Safe OCR Boxes

6. OCR Extraction Layer

Tool

SuryaOCR (Multilingual)

Output Schema

```
{  
  "text": "Applc",  
  "bbox": [x1, y1, x2, y2],  
  "confidence": 0.58  
}
```

This output is **never trusted directly**.

7. Correction Model

7.1 Task Definition

Denoising Auto-Encoding

Given noisy OCR text, generate corrected text **without modifying digits**.

7.2 Model Selection

Primary Model (Recommended)

- **Gemma-2-2B-IT**
 - Small
 - Multilingual
 - PEFT friendly
 - Cost efficient

Optional Comparison Model

- Llama-3.1-8B-Instruct

7.3 Fine-Tuning Method

Aspect	Choice
Method	QLoRA
Precision	4-bit
Adapted Layers	q_proj, v_proj
Optimizer	paged AdamW
Tooling	Unsloth / PEFT

8. Training Data Strategy

8.1 Data Source

- Clean Sinhala text (Wikipedia / news)
- English invoice vocabulary

8.2 Noise Injection (Synthetic)

Noise Type	Simulates
Gaussian blur	Phone photos
Rotation	Handheld skew
Downscale-upscale	Rasterized PDFs
Character swaps	OCR confusion

8.3 Training Sample Format

```
{  
  "instruction": "Correct OCR errors without changing numbers.",  
  "input": "ඉන්වයිසි අංකය Applc 500.00",  
  "output": "ඉන්වයිසි අංකය Apple 500.00"  
}
```

🔴 Digits are identical in input and output

9. Inference-Time Correction Logic

9.1 Correction Unit

- Operates at **line level**
- Preserves bounding boxes
- Applies correction text-only

9.2 Numeric Safe-Guard Algorithm

Rules

A correction is **rejected** if:

1. Digit count differs
2. Digit sequence differs
3. Decimal structure differs

Pseudocode

```
def safe_correct(raw, corrected):
    if extract_digits(raw) != extract_digits(corrected):
        return raw, "raw_ocr"
    return corrected, "ai_corrected"
```

10. Final Box Assembly

Each box contains:

- final text
- original bbox
- confidence
- correction source
- digit lock flag

This enables:

- auditing
- debugging
- trust in finance workflows

11. Quality Evaluation

Metrics

Metric	Purpose
CER	OCR quality
NAR	Numeric integrity
Unsafe blocks	Guard effectiveness

Target

- CER < **5%**
- Numeric Accuracy = **100%**

12. Failure Handling

Failure	Behavior
LLM crashes	Use raw OCR
Unsafe numeric change	Reject correction
Low confidence OCR	Prefer AI text

13. Why This Component Matters

Without Component 1:

- Table reconstruction breaks
- Arithmetic reasoning fails
- Financial answers become unsafe

Component 1 **stabilizes the entire system**.

Summary

Component 1 performs semantic post-correction of OCR output using a PEFT-fine-tuned language model, while enforcing deterministic numeric immutability to ensure financial safety.

Research component 2

Research Component 2

Layout-Aware Spatial Serialization for Multilingual Financial RAG

1. Component Overview

Component Name

Layout-Aware Spatial Serialization Module

Position in System

- Runs **after Component 1** (Semantic OCR Post-Correction)
- Runs **before VectorDB indexing** and before arithmetic reasoning (Component 3)

Core Responsibility

Convert spatially grounded OCR tokens (text + bounding boxes) into **layout-preserving semantic chunks** that:

- Bind values to their labels/headers
- Preserve row-level relationships (line items)
- Are optimized for retrieval accuracy in Sinhala/English financial docs
- Maintain provenance for UI highlighting

2. Problem Definition

2.1 Practical Problem

Invoices, purchase orders, and receipts contain information where meaning is defined by **2D layout**, not sentence flow.

2.2 Failure Mode: Context Loss in Standard RAG

Standard OCR creates a single linear text stream. In financial documents:

- Labels and values may be far apart in raw stream
- Chunking by characters/words breaks label–value association
- RAG retrieves a number without the correct header → LLM guesses → wrong finance answer

Example:

- OCR stream includes: Total Amount ... 50,000 but chunking separates them.
- Retrieval returns 50,000 without “Total Amount”.

3. Design Goals

Goal	Description
Preserve spatial semantics	Keep row/label/value relationships
Retrieval precision	Improve hit rate for “price/qty/total” queries
Robustness	Avoid brittle full-table grid reconstruction
Determinism	No LLM-generated serialization; template-based only
Provenance	Every chunk traces back to page + bbox region
Multilingual	Sinhala/English headers and values

4. Inputs and Outputs

4.1 Inputs (from Component 1)

Required Inputs

- tenant_id (string)
- document_id (string)
- page_images[] (optional but recommended for UI highlighting)
- final_safe_boxes.json (**mandatory**)

final_safe_boxes.json (expected structure)

```
{
  "page": 1,
  "boxes": [
    {
      "text": "Apple",
      "bbox": [120, 300, 200, 335],
      "confidence": 0.58,
      "locked_digits": false,
      "source": "ai_corrected"
    }
  ]
}
```

4.2 Outputs

Primary Output

spatial_chunks.json (exact schema defined in Section 9)

Secondary Outputs

- table_regions.json (optional, if using YOLO/OpenCV to identify table ROIs)
- row_clusters.json (debug artifact)
- header_bindings.json (debug artifact)
- index_manifest.json (what was inserted into VectorDB)

VectorDB Inserts

- chunk_type = "line_item_row"
- chunk_type = "key_value"
- chunk_type = "header"
- chunk_type = "section_text" (optional for non-table content)

5. Internal Architecture

High-Level Flow

Safe OCR Boxes (text + bbox)



(Optional) ROI Detection (table-like zones)



Row Clustering (Y-axis)



Header Detection + Binding (X-axis)



Controlled Semantic Serialization (templates)



Spatial Chunk Generation (JSON)



VectorDB Indexing + Provenance

6. Core Algorithms

6.1 Optional ROI Detection (Table-ish Areas)

Why optional?

If you can reliably cluster rows without table detection, skip YOLO to reduce complexity.

ROI Detection Options

- **Option A (Recommended Lightweight):** OpenCV line detection + contour grouping
- **Option B:** YOLO table region detection for coarse filtering

Output (if enabled)

```
{
  "page": 1,
  "regions": [
    {"region_id": "r1", "type": "table_like", "bbox": [80, 210, 620, 680]}
  ]
}
```

6.2 Row Clustering (Serializer Core)

Objective

Group OCR tokens into “rows” based on **vertical alignment**.

Inputs

- `tokens = [ {text, bbox, confidence, locked_digits} ]`

Key Definitions

- $y_center = (y1 + y2) / 2$
- $text_height = (y2 - y1)$
- $dynamic_y_threshold = median(text_height) * alpha$
 - suggested $alpha = 0.6$ to 1.2 (tune on samples)

Output

Row clusters:

```
{
  "page": 1,
  "rows": [
    {"row_id": "row_001", "tokens": ["t12", "t13", "t14"], "bbox": [...] }
  ]
}
```

6.3 Header Detection

Objective

Identify header rows like:

- Description | Qty | Unit Price | Total

Heuristic Strategy (Deterministic)

Header candidates are rows where:

- Average token confidence is high OR
- Row contains known header keywords (Sinhala + English) e.g.:
 - English: description, qty, quantity, unit price, total, amount, tax, VAT
 - Sinhala: විස්තරය, පරමාණය, ඒකක මිල, මුද්‍ර, බදු, වැට්

Output

```
{
  "page": 1,
  "headers": [
    {"header_id": "hdr_01", "row_id": "row_004", "columns": ["Description", "Qty", "Unit Price", "Total"]}
  ]
}
```

6.4 Header-to-Row Binding (X-axis association)

Objective

Bind each token in a data row to the most likely header column using X-coordinates.

Process

1. Extract header token x-centers:
 - $x_center = (x1 + x2) / 2$
2. For each data token, assign it to nearest header x-center
3. If ambiguous, assign to “unknown_column”

Output (per row)

```
{
  "row_id": "row_007",
  "cells": {
    "Description": "Apple",
    "Qty": "5",
    "Unit Price": "100.00",
    "Total": "500.00"
  }
}
```

6.5 Controlled Semantic Serialization (No Free-Text)

Rule

Serialization must be **template-based** (no generative paraphrasing).

Templates

A) Line Item Row Chunk (preferred)

LineItem | Description: {desc} | Qty: {qty} | UnitPrice: {unit_price} | Total: {total}

B) Key-Value Chunk (for totals, invoice no, date)

KeyValue | {key}: {value}

C) Header Chunk (context)

Headers | {col1} | {col2} | {col3} | ...

7. Chunking Strategy (How many chunks and why)

Primary Strategy: Row-level chunks

- 1 chunk per row for short tables
- For large tables: 5 rows per chunk, but **repeat header** inside each chunk

Decision rule

- If row_count <= 30: one chunk per row
- Else: chunk into blocks of 5–10 rows per chunk

Why

- Avoid embedding dilution
- Improve retrieval specificity (“Apple unit price” maps to one row chunk)

8. Failure Handling & Fallbacks

Failure Case	Fallback
No header detected	produce row chunks with positional ordering only
Sparse/shifted columns	assign unknown fields; preserve token order
Skew causing bad clustering	increase dynamic threshold; optionally deskew upstream
ROI detection fails	skip ROI and run full-page clustering

Important rule:

Even in failure, never drop tokens — emit best-effort chunks + provenance.

9. Exact JSON Schema for Spatial Chunks (Contract)

This is the **canonical artifact** your system will store and index.

9.1 spatial_chunks.json (Top-level)

```
{
  "tenant_id": "sme_001",
  "document_id": "INV_2024_0031",
  "version": "1.0",
  "language_hint": ["si", "en"],
  "pages": [
    {
      "page": 1,
      "chunks": [
        /* SpatialChunk object */
      ]
    }
  ]
}
```

9.2 SpatialChunk Object (Exact)

```
{
  "chunk_id": "ch_000012",
  "chunk_type": "line_item_row",
  "table_id": "t1",
  "row_id": "row_007",
  "header_id": "hdr_01",
  "text": "LineItem | Description: Apple | Qty: 5 | UnitPrice: 100.00 | Total: 500.00",
  "fields": {
    "Description": {"value": "Apple", "token_ids": ["tok_31"]},
    "Qty": {"value": "5", "token_ids": ["tok_32"], "locked_digits": true},
    "UnitPrice": {"value": "100.00", "token_ids": ["tok_33"], "locked_digits": true},
    "Total": {"value": "500.00", "token_ids": ["tok_34"], "locked_digits": true}
  },
  "provenance": {
    "page": 1,
    "bbox": [110, 290, 610, 345],
    "token_bboxes": {
      "tok_31": [120, 300, 200, 335],
      "tok_32": [260, 300, 280, 335],
      "tok_33": [420, 300, 480, 335],
      "tok_34": [540, 300, 600, 335]
    }
  },
  "quality": {
    "struct_confidence": 0.86,

```

```

    "header_bound": true,
    "row_cluster_confidence": 0.90
  },
  "metadata": {
    "currency": "LKR",
    "doc_type": "invoice",
    "source_component": "component_2",
    "created_at": "2025-12-18T05:00:00+05:30"
  }
}

```

Required Fields

- chunk_id
- chunk_type
- text
- provenance.page
- provenance.bbox
- metadata.source_component

Allowed chunk_type values

- "line_item_row"
- "line_item_block" (multiple rows)
- "key_value"
- "header"
- "section_text"

9.3 Token Object Schema (Optional debug file)

If you want a stable ID mapping:

```

{
  "token_id": "tok_31",
  "text": "Apple",
  "bbox": [120, 300, 200, 335],
  "confidence": 0.58,
  "locked_digits": false,
  "source": "ai_corrected"
}

```

10. VectorDB Indexing Rules

What is embedded?

Embed only:

- SpatialChunk.text

What is stored as metadata?

- tenant_id, document_id, page
- chunk_id, chunk_type
- table_id, row_id, header_id

- provenance.bbox
- quality.struct_confidence

This enables:

- tenant isolation filters
- UI row highlighting
- retrieval debugging

11. Evaluation Plan (Research proof)

Baselines

1. OCR linear text + naive chunking (500 chars)
2. (Optional) Full markdown table chunk

Proposed

- Spatial chunks (row-level + header binding)

Metrics

- **Cell-query retrieval hit rate** (primary)
- **Precision@k / Recall@k** for numeric queries
- **Association accuracy**: % of retrieved numbers that match correct header
- **Provenance success rate**: % of answers with valid bbox highlight

One-Sentence Summary (Keep Handy)

Component 2 converts spatially grounded OCR tokens into deterministic, template-based semantic chunks that preserve header–value relationships, improving multilingual financial retrieval accuracy while enabling provenance-level UI highlighting.

Research component 3

Research Component 3

Neuro-Symbolic Arithmetic QA (PAL) for Hallucination-Free Financial Answers

1. Component Overview

Component Name

Neuro-Symbolic Arithmetic Reasoning Module (PAL Layer)

Position in System

- Runs at **query-time** (online)
- Consumes **retrieved spatial chunks** from Component 2
- Uses **candidate-doc scoping** from SQL + Component 4 when document scope is missing

Core Responsibility

Answer finance questions requiring arithmetic with **deterministic correctness**, by:

- Using an LLM to produce a **structured computation plan** (JSON)
- Validating the plan against a strict allow-list
- Executing the plan deterministically (Pandas/SQL)
- Generating a final answer in Sinhala/English using exact computed results
- Returning row-level provenance for UI highlighting

2. Problem Definition

2.1 Practical Problem

LLMs hallucinate arithmetic and label–value associations in financial QA.

2.2 Failure Modes

1. **Arithmetic errors**
LLM predicts a sum incorrectly, especially for many rows.
2. **Field association errors**
“500” could be Unit Price / Total / Discount. LLM guesses.
3. **Scope ambiguity**
User asks “January total spending” without specifying document IDs.

3. Design Goals

Goal	Description
Deterministic arithmetic	100% correct sums/averages/filtering for supported ops
Safe execution	No arbitrary code execution (no raw exec)
Multilingual	Sinhala/English queries produce valid plans
Robust scoping	Works even when doc scope is missing
Explainable provenance	Links results to exact rows + bbox
Production practicality	Fast + auditable + maintainable

4. Inputs and Outputs

4.1 Inputs

A) User query request

```
{
  "tenant_id": "sme_001",
  "language": "si",
  "query": "January පළතුරු සඳහා මුද්‍ර වියදම කීයද?",
  "document_scope": null
}
```

B) Candidate documents (from SQL + Component 4)

Produced by upstream query router:

```
{
  "candidate_docs": [
    {"doc_id": "INV_2025_0012", "issue_date": "2025-01-05"},
    {"doc_id": "INV_2025_0030", "issue_date": "2025-01-21"}
  ]
}
```

C) Retrieved spatial chunks (from Component 2 + VectorDB)

SpatialChunk[] where chunk_type typically includes:

- line_item_row
- key_value (invoice totals)
- header (optional)

Each chunk includes:

- fields (structured values)
- provenance (page + bbox)
- metadata (doc_id, table_id, row_id)

4.2 Outputs

Primary Output

```
{
  "answer_text": "2025 ජනවාරි මාසයේ පළතුරු සඳහා මුද්‍ර වියදම 700.00 LKR යි.",
  "computed": {
    "value": 700.0,
    "currency": "LKR",
    "operation": "sum(line_item.total)",
    "rows_used": [
      {"doc_id": "INV_2025_0012", "table_id": "t1", "row_id": "row_007"},
      {"doc_id": "INV_2025_0030", "table_id": "t2", "row_id": "row_004"}
    ]
  },
  "citations": [
    {"doc_id": "INV_2025_0012", "page": 1, "bbox": [110, 290, 610, 345]},
    {"doc_id": "INV_2025_0030", "page": 1, "bbox": [95, 310, 605, 360]}
  ],
}
```

```

"audit": {
  "plan": { "task": "aggregate_sum", "...": "..."},
  "validation": "passed"
}
}

```

5. Internal Architecture

High-Level Flow

```

User Query
↓
Scope Resolver (SQL + Graph) [if doc_scope missing]
↓
Retriever (VectorDB) [line_item_row chunks]
↓
Planner LLM → JSON Plan
↓
Plan Validator (allow-list + schema)
↓
Deterministic Executor (pandas/sql)
↓
Answer Generator LLM (language-aware)
↓
Response + citations + audit

```

6. Core Submodules

6.1 Scope Resolver (when document_scope is missing)

Purpose

Convert vague queries (“January total...”) into a bounded doc set.

Inputs

- tenant_id
- extracted constraints from query (month/year/vendor/category)

Strategy

1. **Time filter via SQL**
 - Use issue_date or created_at to select January documents.
2. **Graph expansion via Component 4** (optional)
 - Add linked docs (PO ↔ Invoice) for completeness.

Output

- candidate_doc_ids[]

Note: This step happens before PAL planning/execution.

6.2 Planner (Neuro Layer) — JSON Plan Generation

Key Rule

Planner must output a **strict JSON plan**, not Python code.

Plan Types (Supported Tasks)

- aggregate_sum
- aggregate_avg
- aggregate_count
- compare (e.g., “which is higher?”)
- lookup_value (e.g., “unit price of Apple”)
- group_by_sum (e.g., “total spend per vendor”)

Canonical Field Names (Normalization Contract)

Component 3 expects canonical keys:

- item, description
- qty
- unit_price
- total
- tax, discount
- currency
- doc_date, vendor

Component 2 must map Sinhala headers to these canonical fields where possible.

6.3 Plan Validator (Symbolic Guard)

Validation Rules

Reject plan if:

- task not in allow-list
- unsupported operators used
- fields not in canonical field set
- time constraints missing for tenant-wide queries (optional policy)
- ambiguous target field (unit_price vs total) not resolved

Allowed Filter Operators

- eq, in, contains, gte, lte, between

Allowed Aggregations

- sum, avg, count, max, min

Output

- validation_status: pass/fail
- error_reason if fail (feeds retry loop)

6.4 Deterministic Executor

Input

- plan
- row_records[] extracted from SpatialChunk.fields
- optional doc_metadata[]

Execution Approach

- Convert row records into a Pandas DataFrame (safe, deterministic)
- Run filters and aggregations exactly

Output

- exact numeric result
- supporting rows (doc_id/table_id/row_id)

6.5 Answer Generator (Language + Explanation)

Input

- user language (si/en)
- computed result + currency
- citations (bbox references)
- optional: short explanation of computation

Output

- final answer text in Sinhala or English
- optionally: “assumptions used” (e.g., year chosen)

7. Exact JSON Plan Schema (Contract)

7.1 Plan Object

```
{
  "task": "aggregate_sum",
  "scope": {
    "doc_ids": ["INV_2025_0012", "INV_2025_0030"]
  },
  "filters": [
    {"field": "doc_date", "op": "between", "value": ["2025-01-01", "2025-01-31"]},
    {"field": "category", "op": "eq", "value": "fruit"}
  ],
  "measure": {"field": "total", "agg": "sum"},
  "group_by": [],
  "output": {"format": "currency"}
}
```

7.2 Notes

- scope.doc_ids is always provided by Scope Resolver
- If category is not available, planner may fallback to item contains:
 - item in ["Apple","Banana"] or Sinhala equivalents if known
- If year is missing, planner includes:
 - "assumption": "year=2025" (and answer should state it)

8. Retry & Clarification Policy

Retry Loop

If plan validation fails:

1. Send error message back to planner LLM
2. Ask for corrected JSON plan
3. Retry up to 2 times

Clarification (When Necessary)

If ambiguity remains (unit price vs total):

- Ask: "Do you mean unit price or total price?"
OR choose safe default and disclose:
- "Assuming unit price..."

(Your viva will like that you handle ambiguity explicitly.)

9. Provenance & UI Highlighting

Provenance source

From Component 2 chunks:

- doc_id, page, bbox, row_id

In response

Return citations as:

```
{
  "doc_id": "INV_2025_0012",
  "page": 1,
  "bbox": [110, 290, 610, 345]
}
```

UI can highlight exactly which rows contributed to the sum.

10. Evaluation Plan

Baselines

1. **Direct LLM (no tools):** ask model to compute from retrieved text
2. **RAG only:** retrieval + LLM textual answer (no execution)
3. **Proposed PAL:** JSON plan + deterministic executor

Metrics

- **Arithmetic Accuracy Rate:** exact match with ground truth totals
- **Association Accuracy:** correct field selection (unit_price vs total)
- **Plan Success Rate:** % queries where valid plan is produced
- **Latency:** planning + execution time
- **Explainability Score:** % answers with valid row-level citations

Expected result:

- PAL achieves near **100%** arithmetic accuracy for supported operations.

11. Failure Handling

Failure	Fallback
No rows retrieved	broaden retrieval (more doc_ids, lower threshold)
Missing required field	degrade to “best available” (e.g., sum of invoice totals)
Conflicting currencies	respond with per-currency totals + disclose
Unclear month/year	ask year or assume current year and disclose

Summary

Component 3 converts multilingual financial questions into validated structured computation plans and executes them deterministically on structured spatial chunks, guaranteeing arithmetic correctness and providing row-level provenance to prevent numerical hallucinations.

Research component 4

Research Component 4

Multi-Tenant Relationship Index for Cross-Document Retrieval and Reasoning

1. Component Overview

Component Name

Multi-Tenant Relationship Index (MT-RI)

("Graph layer" implemented as a relational relationship index)

Position in System

- Runs ingestion-time after Component 2 has produced spatial_chunks.json
- Runs query-time to expand scope and resolve cross-document links before Component 3 (PAL)

Core Responsibility

Maintain a tenant-isolated relationship index that connects:

- Documents ↔ Vendors
- Documents ↔ Reference Numbers (PO/INV numbers)
- Documents ↔ Documents (e.g., Invoice references PO)
- Optional: Documents ↔ Items / Categories

This solves the "vector DB silo" issue where related documents don't share vocabulary.

2. Problem Definition

2.1 Practical Problem

Vector retrieval treats documents as isolated. Many financial relationships are expressed via:

- Reference IDs (PO numbers, Invoice numbers)
 - Vendor names
 - Document roles (PO vs Invoice)
- ...and not by repeating item descriptions.

2.2 Failure Mode

User asks: "How much did we pay for laptops?"

- PO contains "laptop"
 - Invoice contains "Payment for PO-101" (no "laptop")
- Vector search retrieves PO but misses invoice.

3. Design Goals

Goal	Description
Multi-tenant isolation	Strict tenant_id enforcement on all reads/writes
Production-safe persistence	Postgres as source of truth (no pickle as DB)
Auditable linking	Every edge stores evidence (page, bbox, chunk_id)

Deterministic first	Regex + normalization before fuzzy matching
Conservative entity resolution	Prefer aliases over hard merges when unsure
Query-time usefulness	Fast doc expansion and filtering for Component 3

4. Inputs and Outputs

4.1 Inputs (Ingestion-time)

Required Inputs

- tenant_id (uuid/string)
- document_id (uuid/string)
- doc_type (invoice / purchase_order / receipt / other)
- doc_date (YYYY-MM-DD) (from SQL metadata or extracted key-value)
- spatial_chunks.json (from Component 2)

What Component 4 consumes from Component 2

- chunk_type = key_value (vendor, invoice_no, po_no, date, totals)
- chunk_type = line_item_row (optional for item/category linkage)
- provenance (bbox + page)
- chunk_id for evidence

4.2 Outputs

Primary Output (Persistent)

- Nodes and edges stored in Postgres (entities + links)

Secondary Output (Debug/Audit Artifact)

graph_updates.json

```
{
  "tenant_id": "sme_001",
  "document_id": "INV_2025_0030",
  "entities_created": [
    {"entity_type": "vendor", "canonical_name": "singer sri lanka", "entity_id": "..."}
  ],
  "links_created": [
    {"link_type": "HAS_VENDOR", "from_doc_id": "...", "to_entity_id": "...", "confidence": 0.98}
  ],
  "aliases_added": [
    {"entity_id": "...", "alias_text": "Singer PLC", "score": 0.92, "method": "fuzzy"}
  ]
}
```

Query-time Outputs

- related_doc_ids[]
- filtered_doc_ids[] (by vendor/ref/category)
- edges[] (optional for explainability)

5. Storage Design: Postgres as Source of Truth

Why Postgres

- Concurrency-safe updates
- Easy tenant partitioning
- Indexing for fast lookup
- Stable persistence for SaaS deployment
- Works with your existing metadata DB

NetworkX is optional as an in-memory cache for traversal, but never the source-of-truth.

6. SQL Schema (CREATE TABLE)

Below is a minimal but production-grade schema.

Replace UUID with TEXT if you prefer, but UUID is better.

```
-- ===== ENTITIES =====
-- vendors, reference numbers, items, categories
CREATE TABLE IF NOT EXISTS entities (
  entity_id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  entity_type TEXT NOT NULL,          -- vendor | doc_ref | item | category
  canonical_name TEXT NOT NULL,       -- normalized form
  raw_name TEXT,                     -- first seen raw
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, entity_type, canonical_name)
);
CREATE INDEX IF NOT EXISTS idx_entities_tenant_type
  ON entities (tenant_id, entity_type);
CREATE INDEX IF NOT EXISTS idx_entities_tenant_type_canonical
  ON entities (tenant_id, entity_type, canonical_name);
-- ===== ENTITY ALIASES =====
-- keep fuzzy matches as aliases; do not always hard-merge
CREATE TABLE IF NOT EXISTS entity_aliases (
  alias_id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  entity_id UUID NOT NULL REFERENCES entities(entity_id) ON DELETE CASCADE,
  alias_text TEXT NOT NULL,
  score DOUBLE PRECISION,
  method TEXT NOT NULL,              -- normalize | fuzzy | manual
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_aliases_tenant_entity
  ON entity_aliases (tenant_id, entity_id);
CREATE INDEX IF NOT EXISTS idx_aliases_tenant_alias
  ON entity_aliases (tenant_id, alias_text);
-- ===== DOCUMENT LINKS =====
-- edges between documents and entities or documents and documents
CREATE TABLE IF NOT EXISTS doc_links (
  link_id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
```

```

    from_doc_id UUID NOT NULL,
    link_type TEXT NOT NULL,          -- HAS_VENDOR | REFERENCES | CONTAINS_ITEM |
HAS_REF
    to_entity_id UUID REFERENCES entities(entity_id) ON DELETE SET NULL,
    to_doc_id UUID,                  -- if linking doc→doc
    confidence DOUBLE PRECISION NOT NULL DEFAULT 1.0,
    evidence JSONB,                  -- {page, bbox, chunk_id, extracted_text, rule}
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (
        (to_entity_id IS NOT NULL) OR (to_doc_id IS NOT NULL)
    )
);
CREATE INDEX IF NOT EXISTS idx_links_tenant_from
    ON doc_links (tenant_id, from_doc_id);
CREATE INDEX IF NOT EXISTS idx_links_tenant_to_doc
    ON doc_links (tenant_id, to_doc_id);
CREATE INDEX IF NOT EXISTS idx_links_tenant_type
    ON doc_links (tenant_id, link_type);
CREATE INDEX IF NOT EXISTS idx_links_tenant_to_entity
    ON doc_links (tenant_id, to_entity_id);

```

7. Ingestion-Time Pipeline (Component 4 Logic)

7.1 Extraction Targets

From spatial_chunks.json, extract:

Key Entities

- Vendor name
- Document numbers (INV no, PO no)
- Reference numbers (e.g., “Payment for PO-101”)
- Optional: item names (Apple, Dell Laptop)
- Optional: categories (Fruit) if you have a mapping

7.2 Deterministic Normalization (Stage A)

Always normalize before fuzzy matching.

Vendor normalization rules

- lowercase
- remove punctuation
- remove business suffixes:
 - ltd, pvt, plc, limited, company, co, Sinhala equivalents if known
- collapse whitespace

Example:

- "Singer Sri Lanka (Pvt) Ltd" → "singer sri lanka"

Reference normalization rules

- Extract ref tokens via regex:

- PO patterns: PO[-\s#:]?\d+, Purchase Order[-\s#:]?\d+
- INV patterns: INV[-\s#:]?\d+, Invoice[-\s#:]?\d+
- Normalize:
 - PO-101 → PO_000101
 - INV-31 → INV_000031

7.3 Conservative Fuzzy Aliasing (Stage B)

Only apply fuzzy match after normalization.

Policy:

- If score ≥ 0.92 → attach as alias to existing entity
- If 0.85–0.92 → store as alias but mark confidence lower (no merge)
- If < 0.85 → create a new entity

Important: never delete/overwrite original names; keep provenance.

7.4 Edge Creation

Create edges with evidence:

- HAS_VENDOR: Document → Vendor entity
- HAS_REF: Document → Ref entity (PO_000101)
- REFERENCES: Invoice Document → PO Document if PO exists, else Invoice → Ref entity
- CONTAINS_ITEM: Document → Item entity (optional)

Evidence JSON should include:

- page, bbox, chunk_id
- extracted text snippet
- extraction method (regex/rule)

Example evidence:

```
{
  "page": 1,
  "bbox": [120, 80, 520, 120],
  "chunk_id": "ch_000045",
  "extracted_text": "Payment for PO-101",
  "rule": "regex_po_ref_v1"
}
```

8. Query-Time APIs (Component 4 Services)

Component 4 must expose three query-time functions:

API A: expand_related_docs(doc_ids)

Use case: vector retrieval returns PO, graph expands to invoices referencing it.

Output:

- expanded doc set: original + related docs

API B: filter_docs(tenant_id, vendor=None, ref=None, date_range=None)

Use case: “Singer invoices in January”

API C (Optional): docs_containing_item(tenant_id, item, date_range=None)

Use case: “Apple purchases in January”

Note: If you don’t store CONTAINS_ITEM, you can rely on VectorDB for item filtering.

9. How Component 4 Aligns With Component 3 (No Document Scope)

When user query lacks doc scope:

1. SQL filters candidate docs by date range (January)
2. Component 4 further filters/expands by:
 - vendor
 - PO/INV references
 - linked documents
3. Vector retrieval (Component 2 chunks) runs within this doc set
4. Component 3 computes totals deterministically

This makes tenant-wide queries fast and correct.

10. Evaluation Plan (Research Proof)

Baselines

1. Vector DB only (no graph expansion)
2. Vector DB + naive “search all docs” (no smart linking)

Proposed

Vector DB + Relationship Index expansion

Metrics

- Cross-document recall: % of relevant docs retrieved for linked queries
- Answer correctness uplift: correct answers for queries requiring linked docs
- Expansion precision: how often expansion adds unrelated docs
- Latency: added time from relationship expansion

Test queries:

- “Did we pay for PO-101?”
- “Payments related to Dell laptops?”
- “Singer invoices in January” (vendor filter)

11. Failure Handling

Failure	Behavior
Vendor missing	skip HAS_VENDOR edge
Reference detected but target doc not present	link to ref entity (HAS_REF)
Ambiguous vendor match	store alias, avoid merge
Conflicting ref formats	normalize to canonical, store raw as alias

Summary

Component 4 builds a tenant-isolated relationship index over documents, vendors, and reference identifiers to enable cross-document retrieval and explainable scope expansion, overcoming the vocabulary limitations of pure vector search in financial workflows.